

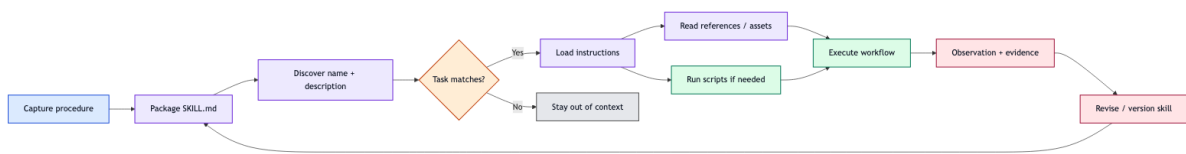
第 18 讲：Agent Skills：把可复用流程做成程序化记忆

Coding Agent Notes (June 2026)

系统位置	skills 位于 context 与 action 中间：它们不是模型权重，也不是低层工具，而是可复用的任务过程。
已知前提	agent 已经具备 repo context、工具调用、运行环境、验证反馈和 long-running session。
新增问题	如何把一套可重复 workflow 做成可发现、可调用、可维护、可审计的程序化记忆。
读完可回答	什么时候应该写 skill，什么时候应该写 tool、plugin、hook、subagent 或 repo instruction。

1 核心图景

Agent skill 最容易被误解成“一个更长的 prompt”。这个理解太浅。prompt 是一次性指令，skill 是一种可复用过程单元：它有名字，有触发描述，有正文步骤，有可选脚本、参考资料和模板；它可以被发现、加载、执行、审计和更新。对 coding agent 来说，skill 不是让模型突然拥有新能力，而是把团队反复使用的工作方法做成可调用的程序化记忆。



这张图里的关键不是文件夹结构，而是生命周期。一个团队先把反复出现的工作过程抽出来，写成 SKILL.md，让 agent 在启动时只看到 name、description 和位置；当任务匹配时，agent 才加载完整步骤，再按需要读取 references/ 或运行 scripts/。执行之后，输出证据和失败反馈又反过来改进 skill。

中心命题：

Skill 是 procedural memory 的工程化形态。它把“每次都要重新教模型的做事方法”变成一个可发现、可版本化、可审计、可复用的工作单元。

这和 #17 的 durable session 不是同一层。durable session 解决的是“同一个任务如何暂停、恢复、回放”；skill 解决的是“同一种任务如何跨会话复用做法”。前者让一次长期工作不断线，后者让下一次类似工作不必从零发明流程。

2 为什么需要 skills

在没有 skill 的系统里，团队通常有三种替代方式。第一，把所有约定塞进 repo-level instruction。它适合长期规则，但不适合大量专门流程，因为这类内容会变成永远加载的背景噪声。第二，把流程写进 prompt。这样短期有效，但每次都要重新粘贴，版本不可控，也很难保证不同人写的是同一套步骤。第三，把流程做成 tool。tool 可以执行动作，但它通常不解释什么时候用、怎么串联、失败后如何恢复。

Skill 的位置正好在这三者之间。它比 ambient context 更按需，比 prompt 更可维护，比 tool 更包含过程知识。一个好的 skill 会回答四个问题：什么时候用，先做什么，再做什么，遇到什么错误如何恢复。它也可以把稳定的机械步骤交给脚本，而把判断留给 agent。

做法	适合内容	不适合内容
一次性 prompt AGENTS.md	临时偏好、当前任务限制 全仓库长期规则、测试命令、编码约定	可复用团队流程 大量按需领域知识
Tool Skill	稳定动作 API、外部系统调用 可复用流程、领域方法、脚本和参考资料	多步骤策略和判断标准 高频低层动作或强权限执行入口
Plugin	分发多个 skills/tools/apps/hooks	单个本地工作习惯

Insight. Skill 的价值不在于“多给模型一段文字”，而在于给可复用流程一个加载边界。没有边界，流程知识要么永远占上下文，要么散落在 prompt、wiki、脚本和个人习惯里。

3 SKILL.md 的最小结构

Agent Skills 标准把 skill 做得很轻：一个目录，至少包含 SKILL.md。这个 Markdown 文件用 YAML frontmatter 写元数据，然后用正文写指令。最小字段是 name 和 description。其中 name 是稳定标识，description 是触发条件和语义索引。

组成	作用	设计要点
name	稳定标识和调用名	短、唯一、和目录一致
description	让 agent 判断何时使用	同时写“做什么”和“何时用”
正文步骤	真正的 procedure	命令式、可执行、少解释性散文
scripts/	可运行代码	自包含、错误消息清楚
references/	按需读取的长文档	单文件聚焦，不做深层链式引用
assets/	模板、图片、数据	只放执行时确实需要的资源

description 是最容易低估的字段。它不是普通摘要，而是 skill 的检索入口。写成“帮助处理 PDF”太宽，模型不知道何时触发；写成“提取 PDF 文本和表格、填写 PDF 表单、合并多个 PDF；当用户提到 PDF、表单或文档抽取时使用”就更可行动。前者像标签，后者像路由规则。

3.1 正文应该像操作规程

Skill 正文不应该写成论文式解释。它应该更接近 runbook：输入是什么，前置检查是什么，步骤是什么，失败时看什么，输出应该长什么样。对 coding agent 来说，一个代码审查 skill 可以规定先看 diff scope，再跑 tests，再按风险分类列 findings；一个 migration skill 可以规定先读 schema，再找 call sites，再写兼容层，再跑回归测试。

如果正文过长，应拆到 references/。原因很简单：一旦 skill 激活，正文会进入上下文。长正文不是免费知识，而是每轮推理都要背的负担。标准建议把详细参考资料放到单独文件，让 agent 在需要时读取。

3.2 Skill 的粒度

Skill 的粒度决定它是否好用。太细的 skill 像一个命令别名，只包装单个动作，例如“运行 formatter”或“打开 issue”。这类动作通常应该做成 tool 或脚本，而不是让模型再读一段流程。太大的 skill 又像

小型手册，把 release、debug、review、security 和 deployment 全塞在一起；它会难触发、难审计，也难判断失败来自哪一步。

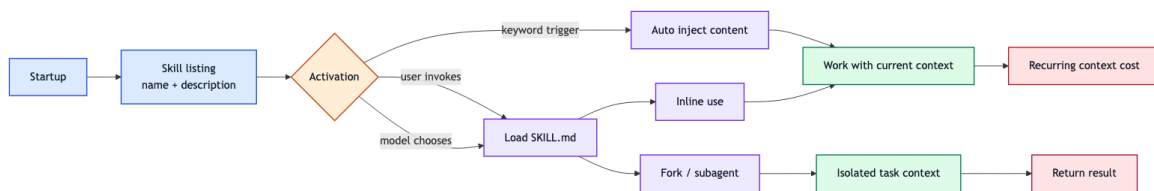
合适的粒度通常有三个特征：任务边界清楚，步骤之间有顺序依赖，输出可以验收。比如“发布一个 Python 包”可以是 skill，因为它包含版本检查、changelog、test、build、upload、tag 和 rollback；“运行 pytest”不适合，因为它只是一个动作；“维护整个后端系统”也不适合，因为它不是一个可执行 workflow。

粒度	典型表现	更合适的归属
过细	单个 shell 命令、单个 API 调用	tool 或 script
合适	多步骤流程，有输入和验收证据	skill
过大	覆盖多个无关任务族	拆成多个 skills 或 plugin
过常驻	每次任务都要遵守	repo instruction
过强制	必须在生命周期点执行	hook

一个实用判断是：如果去掉 skill，agent 会不会每次都重新规划同一套步骤？如果会，而且这套步骤能被 review、能产出证据、能在失败时恢复，那么它值得成为 skill。反过来，如果流程依赖大量临场判断、每次路径都完全不同，先写成普通文档或 checklist 往往更稳。

4 Progressive Disclosure

Skills 的核心机制是 progressive disclosure。启动时 agent 不需要读完整 skill，只需要看到“有哪些 skill”以及“什么时候可能有用”。当当前任务匹配某个 skill 时，再加载完整 SKILL.md。如果正文还不够，再读取 references 或运行 scripts。



这个机制解决了一个长期矛盾：团队希望 agent 随身带很多专业流程，但上下文窗口有限。把所有流程都放进系统 prompt 会污染每次任务；把流程完全藏起来，agent 又不知道它们存在。progressive disclosure 把这个问题拆开：**发现信息常驻，执行信息按需加载**。

4.1 不同产品的实现差异

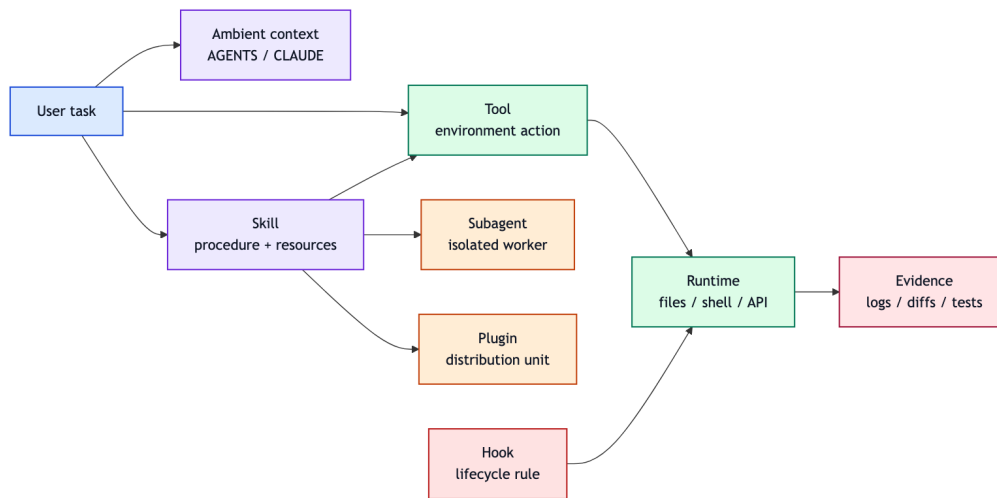
不同系统都在做 progressive disclosure，但细节不同。Codex 手册说明它启动时使用 skill 的 name、description 和路径；选择 skill 后再读完整 SKILL.md。Claude Code 文档强调技能遵循 Agent Skills open standard，同时加入 invocation control、subagent execution 和 dynamic context injection。OpenHands 文档则把 AgentSkills 与 legacy inline skills 分开：legacy content 作为常驻上下文，AgentSkills 只在 available skills 列表中暴露描述，并由 agent 调用。

系统	发现阶段	激活阶段
Agent Skills 标准	name + description	读 SKILL.md，再读资源
Codex	name + description + file path	选择后加载完整指令
Claude Code	skill listing + invocation controls	inline、manual、fork/subagent
OpenHands	<available_skills> 描述列表	invoke_skill() 或 trigger 注入
Gemini CLI	/skills 管理发现状态	enable / disable / reload

Insight. Progressive disclosure 的本质是 context budgeting。它不是为了把知识藏起来，而是让“知道有什么”和“执行具体步骤”占用不同层级的上下文预算。

5 和其他系统面的边界

Skill 必须和其他 agent surface 分清楚，否则系统很快会混乱。一个常见坏设计是：把工具说明写进 skill，把长期规则写进 skill，把权限策略写进 skill，把子任务分派也写进 skill。结果是 skill 变成一个小黑箱，既不可维护，也无法判断失败属于流程、工具、权限还是模型。



更清楚的边界是：ambient context 提供长期背景；skill 提供可复用 procedure；tool 提供环境动作；plugin 负责分发；subagent 负责隔离执行；hook 负责生命周期约束；runtime 返回证据。Skill 可以调用 tool，也可以建议 subagent，但它不应该替代这些边界。

对象	问的问题	典型内容
AGENTS.md	这个 repo 长期怎么工作	测试命令、代码风格、目录约定
Skill	这类任务应该怎么做	release、review、migration、debug 流程
Tool	agent 能对环境做什么	grep、edit、browser、API call
Plugin	如何安装和分发能力	多个 skills、tools、hooks、apps
Subagent	是否需要隔离 worker	research、review、parallel implementation
Hook	何时强制检查或拦截	pre-tool、post-tool、session start

5.1 判断归属的经验法则

如果内容应该每次都生效，放在 ambient context。若内容只在某类任务中生效，放在 skill。若内容是稳定动作接口，做成 tool。若内容需要隔离上下文或并行执行，考虑 subagent。若内容必须在某个生命周期点强制执行，做成 hook。若要分发给别人安装，做成 plugin。

这套规则并不是形式主义。边界清楚，失败才可诊断。一个 skill 触发错了，是 description 问题；一个 tool 返回坏 observation，是 tool schema 问题；一个 hook 拦截过严，是 lifecycle policy 问题。混在一起，所有失败都会被含糊地归因成“agent 不稳定”。

6 Invocation Design

Skill 触发方式至少有三类：模型自动选择、用户显式调用、关键词/规则触发。Claude Code 还支持禁止模型自动调用，让某些 workflow 只能手动触发；也支持 fork/subagent 语义，让复杂任务在隔离上下文里执行。OpenHands 支持 keyword trigger 和 progressive disclosure 的组合。Gemini CLI 则提供 list、reload、enable、disable 等管理命令。

调用方式	适合任务	风险
模型自动选择	明确领域流程，如 API review	description 过宽会误触发
用户显式调用	deployment、release、危险操作	用户需要知道 skill 存在
关键词触发	固定词汇或平台动作	容易被 prompt injection 利用
fork/subagent	长研究、隔离执行、并行处理	需要明确输入和返回格式
禁用自动调用	生产发布、破坏性维护	过于保守会降低可发现性

6.1 触发失败的两种方向

under-trigger 是该用不用。模型明明在做 PDF 表格抽取，却没有加载 PDF skill，结果重新发明流程。常见原因是 description 太抽象、关键词不贴近用户语言、或 skill listing 被预算截断。over-trigger 是不该用却用了。模型看到“deploy”就套 production deployment 流程，实际用户只是问部署概念。常见原因是 description 太宽、when-to-use 缺少边界、或 trigger 只看关键词不看上下文。

因此，写 skill 的第一步不是写步骤，而是写触发条件。好的 description 应该包含正例和边界：使用于什么任务，不用于什么任务。复杂 skill 还应该有关键输入要求，例如 issue id、目标环境、变更范围、允许工具和输出格式。

Insight. Skill 的 description 是路由器，不是广告语。它写得越像营销文案，agent 越难判断何时该用；它写得越像触发条件，skill 越稳定。

6.2 Invocation 需要负例

很多 skill 写得不稳定，不是因为正文差，而是因为它只写了正例。比如“用于 deployment”会让 agent 在讨论部署概念、写部署文档、排查部署事故、真正发布生产版本时都可能触发同一个 skill。更好的写法应该把任务分开：如果是生产发布，要求用户确认目标环境和版本；如果只是解释部署原理，不触发；如果是事故排查，触发 incident skill 而不是 release skill。

负例的作用是给模型一个边界。边界越明确，后续越不需要靠用户纠正。对高风险 skill，负例甚至比正例更重要，因为误触发的成本更高。一个 migration skill 应明确“不用于纯代码重构”；一个 security review skill 应明确“不替代真实安全审计”；一个 release skill 应明确“不在缺少版本号、目标环境和 rollback plan 时继续执行”。

字段	应该写什么	目的
Use when	用户任务满足哪些条件	提高召回
Do not use when	哪些相似任务不该触发	降低误触发
Requires	必须已有的输入和权限	防止 agent 猜前提
Stop when	何时暂停并请求确认	控制风险
Output	必须交付哪些证据	方便 review

7 Skills as Procedural Memory

把 skills 称为 procedural memory，不是比喻。Voyager 在 Minecraft 中使用了一个可增长的 skill library，把成功的动作程序存储起来，在新任务中检索和复用。论文强调这些技能是 executable code，能够表

示 temporally extended、interpretable、compositional behaviors。虽然这是 embodied agent 不是 coding agent，但思想很接近：模型不必每次都从原子动作开始规划，而是可以调用已经验证过的过程片段。

记忆形态	保存的东西	复用方式
Conversation memory	过去对话事实	当前会话继续参考
Vector memory	文档或轨迹片段	检索相似上下文
Repo instruction	长期项目规则	每次任务默认约束
Skill	可执行过程和资源	在匹配任务中加载并执行
Tool schema	环境动作接口	模型调用外部能力

Skill 的特殊性在于它保存的是“如何做”，而不是“知道什么”。一个 API style guide skill 可能包含知识；但它真正有用的部分是 procedure：先找现有 endpoint pattern，再检查 status code，再检查 error body，再更新 tests。这个 procedure 能减少重复搜索、减少风格漂移，也能让新成员和 agent 使用同一套做法。

7.1 从技能库到团队流程库

在 coding agent 场景里，skill library 不一定是模型自动学习出来的。更常见的是团队把高频流程显式写出来：release checklist、migration protocol、code review rubric、incident triage、benchmark reproduction、paper-note pipeline。每个 skill 都是一个小型 workflow artifact。它可以被 review，可以被版本控制，可以随代码库变化更新。

这比把流程存在人的脑子里更可靠，也比把流程散落在 wiki 里更可执行。Wiki 告诉人类怎么做；skill 告诉 agent 何时加载、读哪些文件、运行哪些脚本、输出什么证据。

8 Resources and Scripts

一个成熟 skill 往往不只是一段 Markdown。它可以带脚本、参考文档、模板、样例输出和静态资产。关键原则是：正文保持短，细节放资源，机械步骤放脚本。这样 agent 先读 procedure，再按需要展开细节。

资源	放什么	设计要求
scripts/	lint、转换、抽取、生成、检查脚本	自包含、可重复、错误消息明确
references/	长规范、API 细节、domain notes	聚焦、按需读取、避免深链
assets/	模板、示例图、schema、配置片段	文件名清楚，版本可追踪
metadata	作者、版本、兼容性、依赖	支持审计和迁移
allowed-tools	预期工具边界	注意不同客户端支持差异

脚本尤其重要。自然语言步骤适合表达判断，脚本适合执行稳定机械动作。比如“生成 changelog 草稿”可以让模型判断分类，但 git log 解析、commit 分组、链接生成更适合脚本。这样 agent 的输出更稳定，失败也更容易定位：是脚本错误、输入错误，还是模型判断错误。

8.1 脚本不是权限豁免

Skill 能带脚本并不意味着脚本天然可信。脚本会读写文件、访问网络、调用系统命令，风险可能比普通 prompt 更高。Agent Skills 生态分析也指出，部分 skills 涉及 state-changing 或 system-level actions。实际使用时，脚本必须像代码一样 review：依赖是什么，写哪些路径，是否访问网络，是否泄露数据，是否可回滚。

9 Worked Example: 论文笔记生产 skill

为了让 skill 的设计更具体，可以看一个接近本系列工作的例子：把“读一组论文并产出一篇可发布 notes”写成 skill。这个任务不是单个动作。它包含下载资料、抽取文本、建立 source card、整理 claims、写讲义、渲染图、编译 PDF、视觉检查和最终归档。每一步都可复用，但每篇论文的内容又不同，所以它很适合 skill。

一个坏版本会这样写：

```
---
name: paper-notes
description: Help write paper notes.
---
Read papers and write good notes.
Make it clear and polished.
```

这个版本几乎没有工程价值。它没有说什么时候触发，没有规定输入是什么，没有说明要下载哪些 source，没有要求 claim tracing，也没有告诉 agent 如何判断 PDF 是否达标。模型读到它以后，仍然只能靠临场发挥。

更好的版本应该把触发、输入、步骤、停止条件和输出证据都写出来：

```
---
name: technical-note-production
description: Use when creating a Chinese technical
  note from papers, repos, docs, or reports. Do not
  use for short summaries or casual Q&A.
---
1. Collect sources and create sources.md.
2. Extract claims into source_digest.md.
3. Build an outline around one self-contained thesis.
4. Write notes.tex with diagrams, insight boxes,
  tables, limitations, takeaway, and references.
5. Render Mermaid diagrams externally with colors.
6. Compile PDF and require more than 10 pages.
7. Run visual QA and text-artifact scan.
8. Copy PDF, TeX, and QA sheet to outputs/.
```

这个例子仍然不完美，但已经有了 workflow 的形状。它告诉 agent：这不是“写摘要”，而是“生产讲义”；不是“看到 paper 就触发”，而是在用户明确要求制作 technical note 时触发；不是只交一段文字，而是要交 PDF、TeX、source tracing 和 QA evidence。

9.1 把细节放到资源里

真正生产环境里的这个 skill 不应该把所有细节都塞进正文。正文只保留流程骨架，长规范放进 references/，机器检查放进 scripts/，模板放进 assets/。例如：

文件	内容	何时读取
SKILL.md	触发条件、主流程、停止条件	skill 激活后立即读取
references/style.md	标题、图表、引用、页数偏好	写作和排版阶段读取
references/qa.md	PDF 检查清单和坏例子	编译后读取
scripts/check_figures.py	检查 Mermaid 颜色和宽度	渲染后运行
assets/template.tex	LaTeX 模板	新建 note 时复制

这样做的好处是 progressive disclosure 更干净。agent 先看到“这是一套 note production workflow”，需要写作时才读 style，需要验收时才读 QA，需要新建文件时才读模板。它不需要在任务一开始就把所有规范塞进上下文。

9.2 这个例子的验收标准

一个 paper-note skill 的验收不应该只看“是否生成了 PDF”。更合理的验收是多维的：

- 内容上，notes 要围绕一个自洽命题展开，而不是论文段落的机械摘要。
- 证据上，关键事实要能回到 source card 或 source digest。
- 视觉上，Mermaid 图要有颜色，宽高比合理，方图不能被拉成满宽。
- 篇幅上，正式 note 必须超过 10 页，且新增页数来自解释深度而不是空白。
- 维护上，PDF、TeX、source files、QA sheet 要进入固定输出目录。

这个例子说明 skill 的真正产物不是一段 prompt，而是一套可重复生产 artifact 的小型系统。它让不同 agent、不同时间、不同 topic 仍然遵守同一套工作方法；同时每篇 notes 的内容仍然来自具体资料，不会变成模板化复读。

10 Management and Distribution

当 skill 只是个人文件时，管理很简单；当团队、仓库和组织都开始使用 skills，管理就变成系统问题。Codex 支持 repository、user、admin、system 等不同来源。OpenHands 支持 installed skill lifecycle，可以 list、load、enable、disable、uninstall。Gemini CLI 有 /skills 命令用于 list、reload、enable、disable。Claude Code 提供 skill listing、manual invocation、配置和调试入口。

这些管理面说明 skill 已经不是孤立 prompt 文件，而是一种可运维 artifact。

管理动作	为什么需要	如果缺失会怎样
List	知道当前 agent 可用技能	用户不知道能力边界
Reload	修改后即时生效	迭代 skill 需要重启环境
Enable/disable	控制风险和噪声	无法隔离坏 skill
Version	追踪流程变化	不知道哪个流程产生结果
Distribute	跨 repo / 团队复用	每个项目复制粘贴
Audit	检查脚本和权限	第三方 skill 风险不可见

10.1 Plugin 是分发边界

Codex 手册把 skills 和 plugins 分开：skill 是 reusable workflow 的 authoring format，plugin 是 installable distribution unit。这个区分很重要。单个 repo 内部流程可以只是 skill；多个 skills、工具、MCP 配置、hooks 或 app 需要一起安装时，才应该做 plugin。过早插件化会增加维护成本；过晚插件化会导致复制粘贴和版本漂移。

10.2 版本治理和退役

Skill 一旦被团队使用，就会产生版本问题。代码库目录会变，测试命令会变，CI provider 会变，依赖管理方式也会变。如果 skill 没有 owner、版本和更新时间，它会慢慢变成“看似权威的旧流程”。这比没有 skill 更危险，因为 agent 会更有信心地执行过期步骤。

因此，成熟 skill 应该至少记录四类元信息：适用范围、维护者、最后验证时间、依赖假设。适用范围回答它绑定哪些 repo、语言、框架和工具；维护者回答谁负责改；最后验证时间回答它最近一次在真实任务中通过是什么时候；依赖假设回答它依赖哪些命令、权限、目录和外部服务。

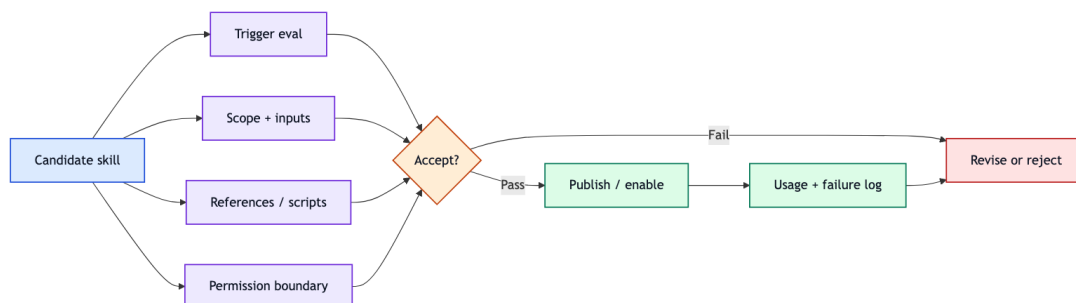
治理动作	触发条件	输出
Smoke test	repo 结构或命令变化	更新步骤或标记失效
Version bump	流程语义变化	changelog 和兼容性说明
Deprecate	被新 skill 替代	指向新 skill，停止自动触发
Split	skill 覆盖太宽	多个更窄的 skills
Merge	多个 skill 高度重复	canonical skill

退役同样重要。旧 skill 不应该悄悄留在 listing 里继续竞争触发。更好的做法是先降低触发权重或禁用自动调用，再在正文顶部写清替代 skill，最后从分发包里移除。这样可以避免“两个相似流程同时存在，模型随机选一个”的问题。

11 Ecosystem and Safety

Agent Skills 的生态正在快速变大。Agent Skills 数据分析论文研究了 40,285 个公开 listed skills，发现 software engineering workflows 高度集中，也发现 redundancy、supply-demand imbalance 和 safety risks。这个结论对 coding agent 特别相关，因为很多 skills 正是围绕环境配置、DevOps、代码修改、仓库操作和系统命令写的。

风险	典型表现	缓解方式
重复和低质量	多个 skill 做同一件事，质量不明	canonical skill、版本、评分和维护者
过宽触发	description 太泛，任务不相关也加载	trigger eval、负例、手动调用
过长正文	激活后持续占上下文	拆 references，压缩正文
危险脚本	删除、提交、发布、系统配置修改	least privilege、review、sandbox
上下文污染	skill 夹带不可信指令或外泄要求	来源审计、签名、隔离执行
陈旧流程	repo 改了，skill 仍按旧结构操作	smoke test、owner、更新周期



skill quality gate 应该像代码 review 一样严肃。尤其是第三方 skills：它们既是 instruction，又可能带脚本和资源。一个恶意或粗糙 skill 不需要攻破模型，只要在“什么时候用”和“如何做”里植入错误步骤，就可能让 agent 在真实 repo 中做错事。

11.1 安全边界要和能力边界一起设计

不要等到 skill 写完再补安全。写 skill 时就应该回答：它需要哪些工具？是否会写文件？是否会运行 shell？是否会读 secrets？是否会访问网络？失败时是否会回滚？输出是否包含可审计证据？这些问题决定 skill 是否应自动触发，是否应手动调用，是否应 fork 到 subagent，是否需要 hook 或 permission gate。

Insight. Skill 是能力，也是攻击面。它把流程变得可复用，也把错误流程、过期流程和恶意流程变得可复用。

12 Acceptance Checklist

一个 skill 写完后，不应该只问“能不能触发”。至少要做六类验收。

验收项	通过标准	失败信号
触发精度	正例会触发，负例不触发	under-trigger / over-trigger
输入契约	必要参数、路径、环境清楚	agent 先猜前置条件
步骤可执行	每一步有明确动作和停止条件	流程像建议，不像规程
资源边界	references/scripts/assets 按需加载	正文过长或深层引用链
权限边界	所需工具、写入、网络明确	脚本隐式扩大权限
证据输出	产出 diff、log、test、summary	只说完成，无法 review

最小 eval 可以很简单：准备三个应该触发的任务、三个不应触发的任务、一个缺少输入的任务、一个脚本失败的任务。观察 agent 是否选择 skill，是否按步骤执行，是否在缺输入时询问，是否在脚本失败时停止并给出可恢复信息。这个 eval 不一定要自动化，但必须存在。

12.1 何时把 skill 升级成别的东西

Skill 不是所有复用的终点。如果 skill 主要是在调用同一个外部 API，应该把 API 做成 tool。如果 skill 包含多个工具、hooks、MCP server 和多个 workflows，应该做 plugin。如果 skill 每次都要跑长研究或并行任务，应该考虑 subagent。如果 skill 的关键是“在每次工具调用前强制检查”，应该做 hook。如果 skill 是所有任务都需要的 repo 事实，应该移到 ambient instruction。

这个升级判断能防止 skill 变成万能袋。万能袋越大，越难触发、越难审计、越难迁移。

13 局限与开放问题

第一，skill selection 还不是已解决问题。description 是弱路由器，容易受措辞、预算、相似 skill 和用户输入影响。大量 skills 同时存在时，listing budget、排序和截断策略会影响模型看见什么。

第二，skill portability 有天然边界。Agent Skills 标准定义了基本格式，但不同产品对 allowed-tools、subagent、trigger、dynamic injection、插件分发、权限确认的支持不同。一个 skill 在一个客户端里安全，在另一个客户端里未必有同样的权限边界。

第三，skill marketplace 的信任机制还不成熟。生态分析已经显示重复、供需错配和高风险动作存在。未来需要更明确的 signing、versioning、risk labeling、static scanning、runtime sandbox 和社区治理。

第四，skill 自动学习仍然困难。Voyager 展示了可增长 skill library 的潜力，但 coding agent 要自动沉淀 skill，需要判断哪些轨迹真正成功、哪些步骤可泛化、哪些 repo-specific 细节应该保留或删除。这接近 program synthesis、workflow mining 和安全审计的交叉问题。

14 最终 Takeaway

Agent skills 可以概括成三句话。

- Skill 是可复用 procedural memory，不是更长 prompt。
- 好 skill 的核心是 progressive disclosure：轻量发现，按需加载，资源和脚本延迟展开。
- Skill 必须有边界：触发边界、资源边界、工具边界、权限边界、输出证据边界。

因此，真正的问题不是“要不要写 skills”，而是“哪些流程值得变成可复用 artifact”。高频、稳定、可审计、有明确输入输出的流程适合 skill；低层动作适合 tool；长期规则适合 ambient context；强制检查适合 hook；可安装能力包适合 plugin。把这些边界分清楚，skills 才能成为 coding agent 的生产力层，而不是又一个上下文噪声来源。

参考资料

- [1] [Agent Skills Overview](#). definition, folder structure, progressive disclosure
- [2] [Agent Skills Specification](#). frontmatter fields, resources, validation, disclosure budget
- [3] [Claude Code Skills](#). invocation control, subagent execution, dynamic context injection
- [4] [OpenHands Agent Skills & Context](#). prompt placement, triggers, installed-skill lifecycle
- [5] [Codex Manual: Agent Skills](#). Codex skill discovery, locations, budget, plugins
- [6] [Gemini CLI Commands](#). /skills management surface
- [7] [Voyager](#). executable-code skill library as reusable procedural memory
- [8] [Agent Skills: A Data-Driven Analysis](#). skill ecosystem, redundancy, and safety risks